AGENT EVOLUTION · NOTEBOOK 2 OF 8

# Version 2 — Give the Agent Hands: PostgreSQL Tool Calling

A guide to `agent_postgres_tool.ipynb`: what a tool is, how the agent loop runs it, what happens inside `run_query`, and how errors become the next step of the loop.

PA

**Tutor: Prudhvi Akella**
Lead Data & AI Engineer · 13+ years in data engineering and AI architecture

Stack: LangChain `create_agent` · OpenAI chat model · psycopg 3 · PostgreSQL  Tools: `run_query(sql)`

1. What changes from Version 1
2. The end-to-end workflow
3. What a tool is
4. The graph, now with a tools node
5. The agent loop, all paths open
6. One call, step by step
7. Inside `run_query`
8. Errors are observations
9. Layers of safety
10. What changed in the prompt
11. Reading the trace and the answer
12. The limitation, and what Version 3 changes
13. Key takeaways

## 1 What changes from Version 1

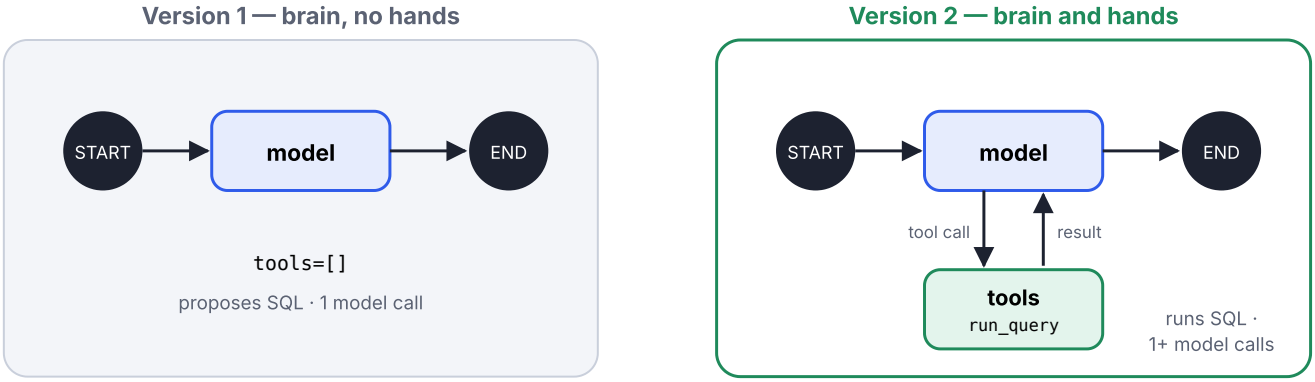



Figure 1. One argument changes: `tools=[]` becomes `tools=[run_query]`. The graph gains a `tools` node and a loop.

| | Version 1 | Version 2 |
|---|---|---|
| Tools | `[]` | `[run_query]` |
| Execute SQL | No | Yes, read-only, through the tool |
| See real results | No | Yes, up to 50 rows |
| Model calls per question | Exactly 1 | 1 or more (one per loop pass) |
| Prompt tells it to | propose SQL | run SQL and answer from the result |

| | | |
|---|---|---|
| Output sections | QUESTION · PROPOSED SQL · EXPLANATION · ASSUMPTIONS | ANSWER · DENOMINATOR · SOURCE · CAVEAT |

The agent is the same agent. The model, the `create_agent()` call and the prompt anatomy all carry over. What is new is one Python function the model can ask to run.

## 2 The end-to-end workflow

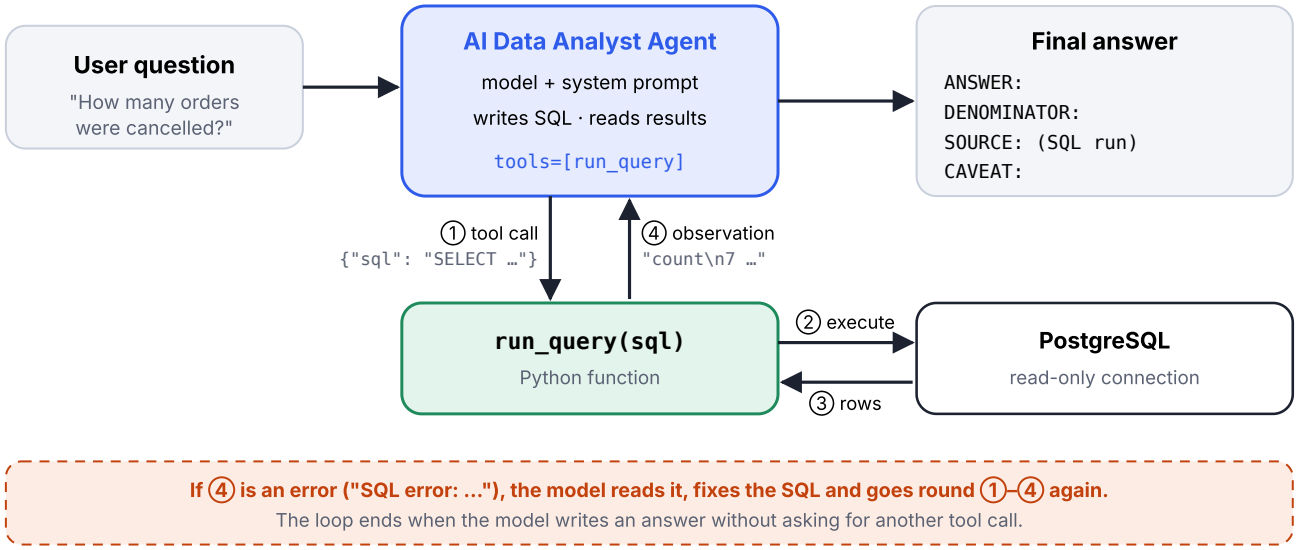



Figure 2. Version 2 end to end. The model asks; Python runs the SQL; the result comes back as an observation.

1. **The user asks a question.** "How many orders were cancelled?"
2. **The model decides it needs data.** Instead of writing an answer, it replies with a **tool call**: the tool name `run_query` and the argument `sql`.
3. **The agent runs the tool.** `run_query` opens a read-only PostgreSQL connection, runs the SQL and formats up to 50 rows as text.
4. **The result goes back to the model** as an observation.
5. **The model reads the observation** and either answers or asks for another query.

> **The model never touches PostgreSQL.** It only writes text that says "please run `run_query` with this SQL". Python does the work and hands back the result. This split is what makes it possible to put safety checks in the tool.

## 3 What a tool is

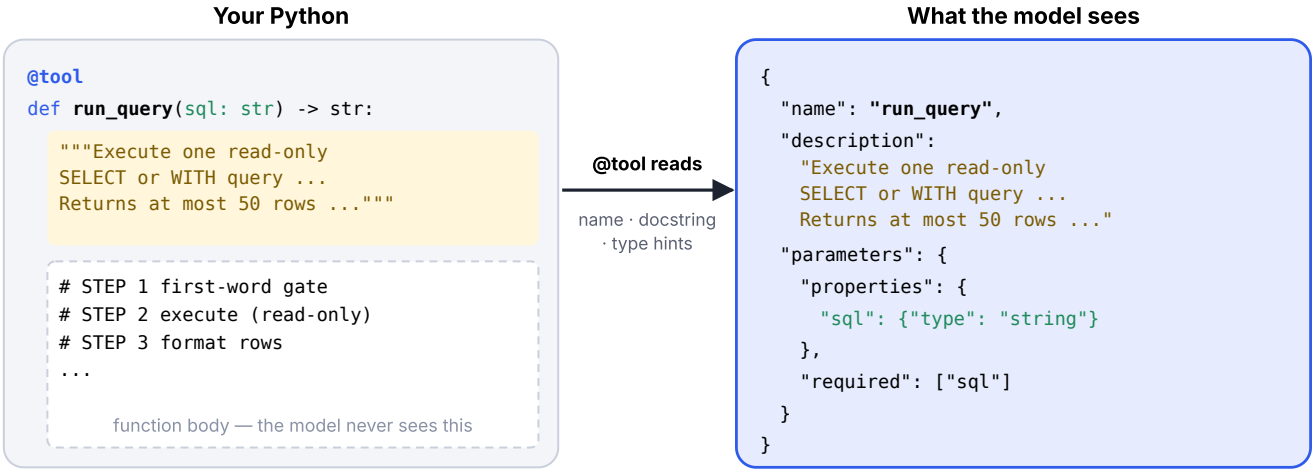



Figure 3. The `@tool` decorator turns a Python function into a schema. This is the real schema LangChain builds for `run_query`.

A **tool** is a Python function plus a description of how to call it. The `@tool` decorator builds that description from three things:

| From the function | Becomes | Used by the model to... |
|---|---|---|
| Function name | `"name": "run_query"` | say which tool it wants |
| Docstring | `"description"` | decide **when** to call it and what to expect back |
| Type hint `sql: str` | `"parameters"` | know which arguments to send, and their types |

When the agent calls the model, it sends this schema along with the messages. The model never sees the function body. Everything it knows about the tool comes from the name, the docstring and the parameters.

> **The docstring is written for the model, not for developers.** "Returns at most 50 rows ... or a message starting with `SQL error:` or `REFUSED:` " tells the model what results look like, so it knows how to read an error and retry. A vague docstring gives the model a vague idea of the tool.

## 4 The graph, now with a tools node

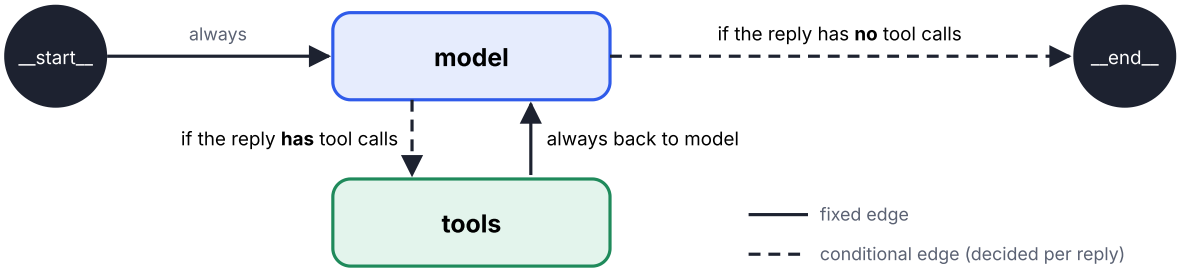



Figure 4. The Version 2 graph. The dashed edges are chosen each time the model replies.

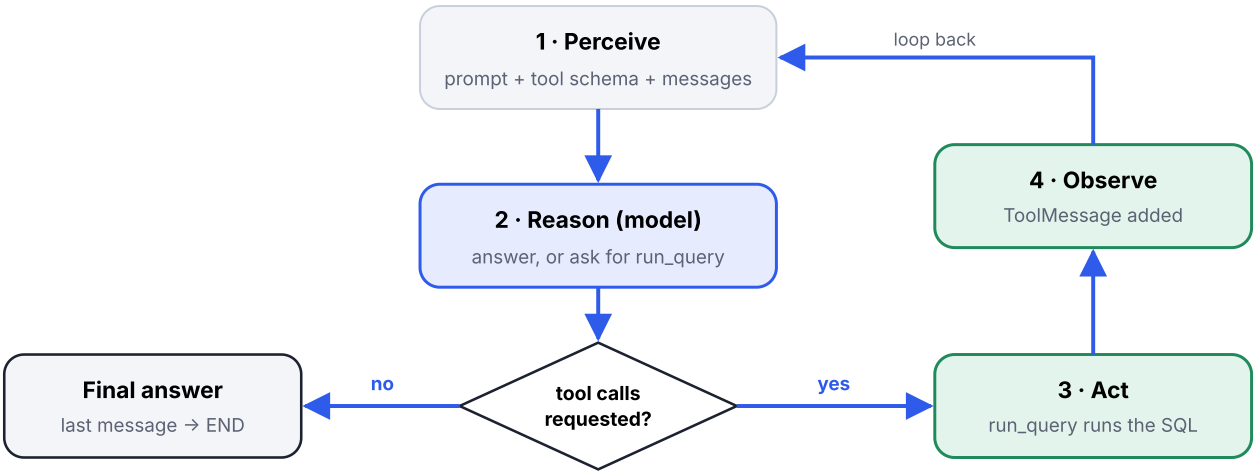
With one tool passed in, `create_agent()` builds a graph with four nodes. These are the real nodes and edges from the notebook's agent:

```
agent.get_graph().nodes  → ['__start__', 'model', 'tools', '__end__']

edges:  __start__ → model      fixed
        model     → tools      conditional   (reply has tool calls)
        model     → __end__    conditional   (reply has no tool calls)
        tools     → model      always back
```

The `tools` node runs every tool call in the model's reply and adds each result to the messages. Then control always goes back to `model`. Only the model can end the run, by replying without a tool call.

## 5 The agent loop, all paths open




Typical run: pass 1 → yes (run_query) → pass 2 → no (answer). With one SQL error: 3 passes.

Figure 5. The same loop as Version 1. Now the "yes" branch exists, so the loop can go round.

The loop has the same four steps as in Version 1. The difference is that steps 3 and 4 can now happen.

1. **Perceive:** the model receives the system prompt, the `run_query` schema and all messages so far, including earlier tool results.
2. **Reason:** the model either writes the answer or asks for `run_query` with some SQL.
3. **Act:** the `tools` node runs `run_query`.
4. **Observe:** the result is added as a `ToolMessage`. Back to step 1.

| Situation | Model calls (passes) | Why |
|---|---|---|
| SQL works the first time | 2 | pass 1 asks for SQL; pass 2 answers |
| First SQL fails, second works | 3 | the error is read and fixed in pass 2 |
| Model needs two separate figures | 3 or more | one query per figure, then the answer |
| Question needs no data (e.g. "what is SQL?") | 1 | answers directly, no tool call |

> Every pass is a full model call, with the whole message list sent again. More passes mean more time and more tokens. Version 2 has no limit on passes; Version 6 adds one for SQL calls.

## 6 One call, step by step

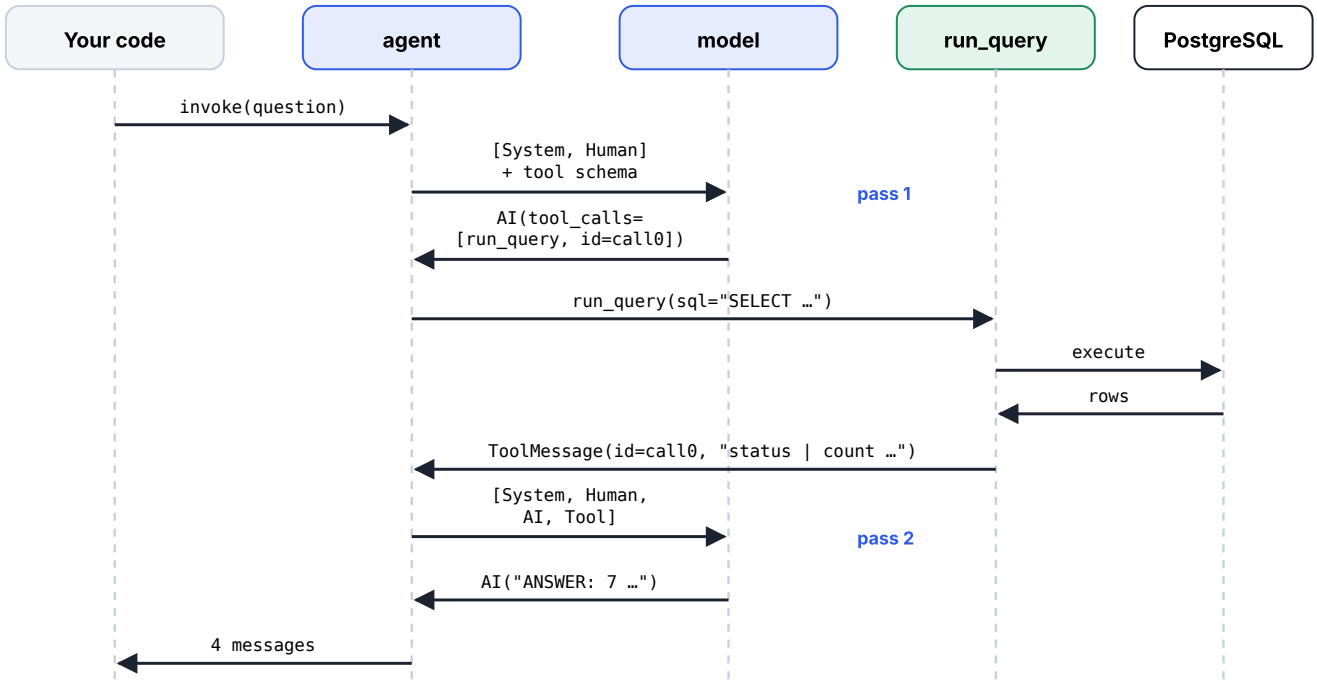



Figure 6. One `ask_agent()` call with a single successful query: two model passes, one tool run.

The real message list after *"How many orders were cancelled?"*. The SQL and numbers are example values:

```
result["messages"] == [
  HumanMessage("How many orders were cancelled?"),

  AIMessage(content="",
            tool_calls=[{"name": "run_query",
                         "args": {"sql": "SELECT status, COUNT(*) FROM orders GROUP BY 1"},
                         "id":   "call0"}]),                  ← pass 1: the model asks

  ToolMessage(content="status | count\ncancelled | 7\n(1 rows.)",
              tool_call_id="call0", name="run_query"),          ← the tool's result

  AIMessage(content="ANSWER: 7 orders ...", tool_calls=[])       ← pass 2: the answer
]
```

### What to notice

- **The first AIMessage has empty `content`.** When the model asks for a tool, it usually writes no text. The request is in `tool_calls`.
- **`tool_call_id` links result to request.** The `ToolMessage` carries `call0`, the same id as the request. If the model asks for two tools in one reply, the ids keep the results apart.
- **The tool result is plain text.** `run_query` returns a string; the model reads it like any other message.
- **The system prompt is still not in the list.** The agent adds it on every model call, as in Version 1.
- **The last message is the answer,** so `result["messages"][-1].content` still works.

# 7 Inside `run_query`

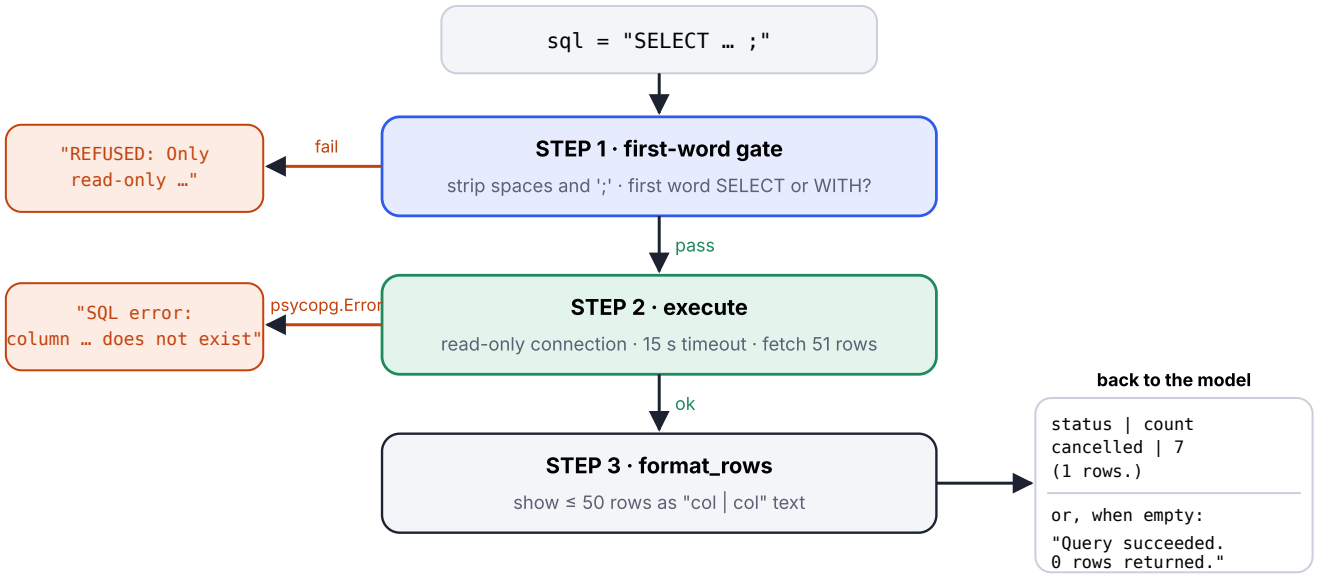



Figure 7. `run_query` always returns text. It never raises an exception into the agent loop.

## 7.1 Every outcome is a string

There are four possible results, and all of them are text the model can read:

| Outcome | Text returned |
|---|---|
| Gate refuses the SQL | `REFUSED: Only read-only SELECT or WITH queries are allowed.` |
| PostgreSQL raises an error | `SQL error: column "order_status" does not exist` |
| Query returns no rows | `Query succeeded. 0 rows returned.` |
| Query returns rows | a header line, one line per row, and a row-count line |

If the tool raised a Python exception instead, the run would stop and the model would get no chance to fix its SQL. Returning the error as text turns a failure into the next step of the loop (section 8).

## 7.2 The connection: `pg_connect()`

```
options = "-c default_transaction_read_only=on -c statement_timeout=15000"
```

| Setting | Effect |
|---|---|
| `default_transaction_read_only=on` | every transaction on this connection is read-only; PostgreSQL rejects writes |
| `statement_timeout=15000` | PostgreSQL cancels any statement running longer than 15 seconds |
| `connect_timeout=10` | give up if the server does not answer within 10 seconds |
| `row_factory=dict_row` | each row comes back as a dict, e.g. `{"status": "cancelled", "count": 7}` |

A new connection is opened for every call and closed when the `with` block ends.

### 7.3 Knowing when rows were cut off

```
MAX_ROWS = 50
fetched = cursor.fetchmany(MAX_ROWS + 1)   # ask for 51
rows    = fetched[:MAX_ROWS]                # show 50
truncated = len(fetched) > MAX_ROWS         # was there a 51st?
```

| Rows the query matches | Fetched | Shown | Last line of the result |
|---|---|---|---|
| 7 | 7 | 7 | `(7 rows.)` |
| 50 | 50 | 50 | `(50 rows.)` |
| 60 | 51 | 50 | `(50 rows shown. More rows exist; the result was capped at 50.)` |

Fetching one extra row is the cheap way to know whether more exist, without counting them all. The last line tells the model the table is incomplete, so it does not treat 50 rows as the full answer.

> The cap limits the rows **returned**, not the work the database does. `SELECT * FROM orders` on a large table still makes PostgreSQL scan it. Good SQL (filters, `LIMIT`, aggregation) is still the model's job.

# 8 Errors are observations

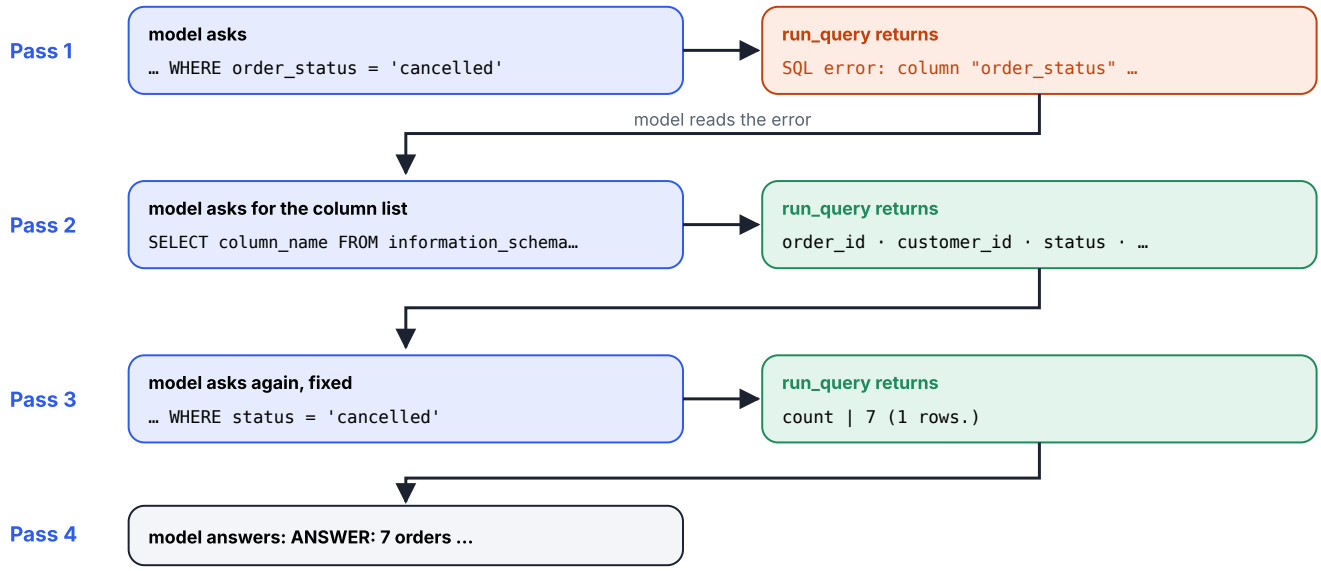



Figure 8. An illustrative error-and-repair run. Column names and numbers are example values.

Version 2 still gives the model only table names, so its first guess at a column can be wrong. Because `run_query` returns the error as text, the model sees exactly what failed and can try again.

1. **Pass 1:** the model guesses `order_status`. PostgreSQL replies that the column does not exist.
2. **Pass 2:** the model may look the columns up itself. A query on `information_schema.columns` is a plain `SELECT`, so the gate allows it.
3. **Pass 3:** with the real column name, the query works.
4. **Pass 4:** the model answers from the result.

This works, but it is slow and wasteful. Each wrong guess costs a full model pass and a database round trip. The prompt rule “If SQL fails, use the error to correct and retry when possible” is what makes the model try to repair the query instead of giving up.

> **Good error messages are part of the tool design.** `SQL error: column "order_status" does not exist` names the exact problem. A message like “query failed” would leave the model guessing again.

## 9 Layers of safety

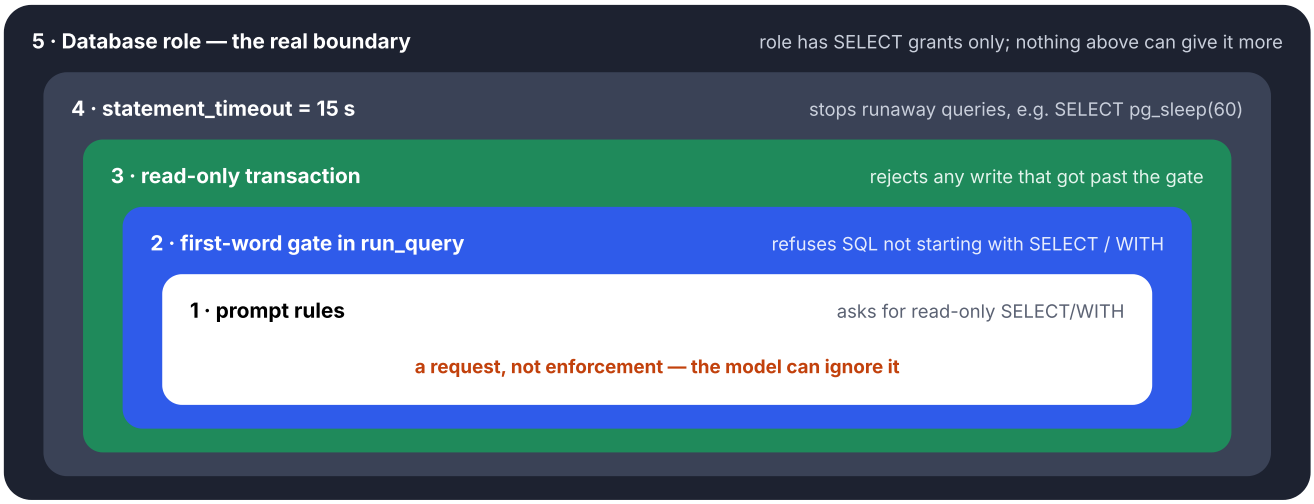



Figure 9. Each layer catches what the one inside it misses. Only the outer layer is a true security boundary.

| SQL the model sends | Stopped by | What the model sees |
|---|---|---|
| `DROP TABLE orders` | 2 · gate | `REFUSED: Only read-only SELECT or WITH queries are allowed.` |
| `WITH x AS (DELETE FROM orders RETURNING *) SELECT * FROM x` | 3 · read-only | `SQL error: cannot execute DELETE in a read-only transaction` |
| `SELECT 1; DELETE FROM orders` | 3 · read-only | starts with `SELECT`, so the gate passes it. It comes back as a `SQL error:`, and the `DELETE` never runs. |
| `SELECT pg_sleep(60)` | 4 · timeout | `SQL error: canceling statement due to statement timeout` |
| anything the layers above miss | 5 · role | `SQL error: permission denied …` |

The first-word gate is weak on purpose in this version. It only looks at the first word, so a write hidden inside a `WITH`, or after a `;`, gets through it. The read-only transaction is what catches those.

> **Never rely on the prompt or the gate alone.** The notebook says it plainly: “A prompt is not a security boundary.” Connect with a database user that has only `SELECT` permission. Version 4 replaces the first-word gate with a proper SQL policy.

## 10 What changed in the prompt

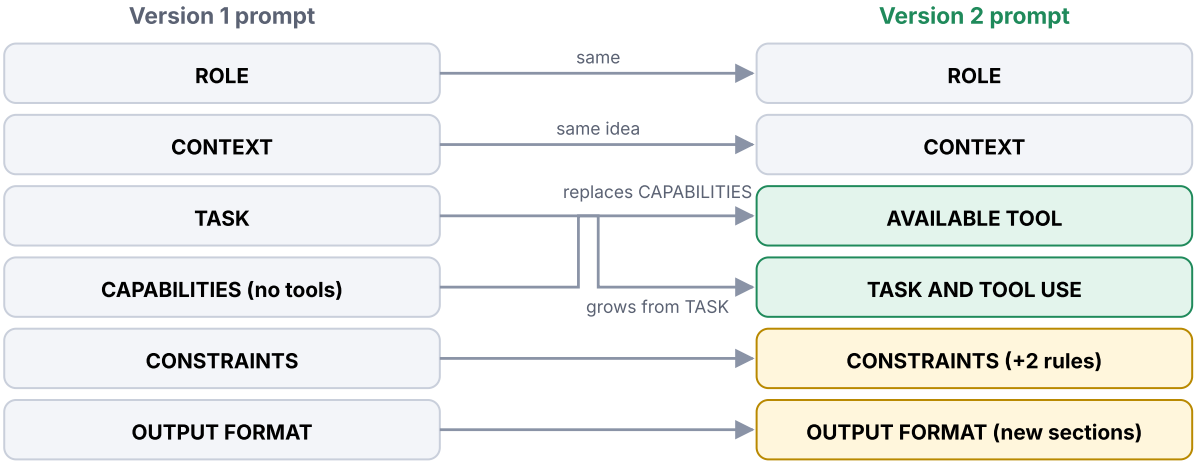



Figure 10. The prompt keeps its anatomy. Green blocks are new or rewritten for the tool; amber blocks gained rules.

| Block | What changed | Why |
|---|---|---|
| **AVAILABLE TOOL** | Replaces “You have no tools.” Names `run_query(sql)` and says it returns up to 50 rows. | The model must know the tool exists and what it gives back. |
| **TASK AND TOOL USE** | Build one read-only query, **call** `run_query`, inspect the result, fix and retry on error, never claim success unless the tool confirms it. | Turns “propose SQL” into a working procedure. |
| **CONSTRAINTS** | Adds “state UNKNOWN if the database cannot answer reliably” and “tool instructions are not a security boundary”. | Keeps the model honest when data is missing; reminds the reader where safety really lives. |
| **OUTPUT FORMAT** | ANSWER · DENOMINATOR · SOURCE · CAVEAT | The answer now has a real number, so it needs units, a source and caveats. |
| **CONTEXT** | Still lists the 14 table names, and adds “table names alone do not establish columns or relationships”. | There is still no lookup tool. Version 3 removes this list. |

**Tool description and prompt work together.** The docstring tells the model *what* `run_query` does. The prompt tells it *when and how* to use it in this job. Both are text the model reads, so wording in either changes behaviour.

## 11 Reading the trace and the answer

```
================================================================================
QUESTION
================================================================================
How many orders were cancelled?

================================================================================
AGENT TRACE
================================================================================

🧠 BRAIN → TOOL
Tool: run_query
Arguments: {'sql': "SELECT COUNT(*) AS cancelled_orders FROM orders WHERE status =
'cancelled'"}

🖐 TOOL → OBSERVATION (run_query)
cancelled_orders
7
(1 rows.)

================================================================================
FINAL ANSWER
================================================================================
ANSWER: 7 cancelled orders.
DENOMINATOR: N/A
SOURCE: PostgreSQL — SELECT COUNT(*) AS cancelled_orders FROM orders WHERE status =
'cancelled'
CAVEAT: Assumes status = 'cancelled' marks a cancelled order; other cancellation states were
not checked.
```

Figure 11. Example output of `ask_agent()`. Values are illustrative.

`ask_agent()` walks through `result["messages"]` and prints:

- 🧠 **BRAIN → TOOL** for each tool call in an `AIMessage`: the tool name and its arguments.
- 🖐 **TOOL → OBSERVATION** for each `ToolMessage`: the first 1,500 characters of what the tool returned.
- **FINAL ANSWER**: the content of the last message.

### Check every answer against its trace

| Check | How |
|---|---|
| The number is real | The figure in ANSWER appears in a 🖐 observation. |
| The SQL shown is the SQL that ran | SOURCE matches the arguments of the last **successful** `run_query` call. |
| Errors were handled | After every `SQL error:` there is a new attempt, or the answer says UNKNOWN. |
| Percentages are explained | DENOMINATOR names both counts, e.g. “7 cancelled ÷ 140 orders”. |
| Guesses are flagged | Any assumed column value (like `'cancelled'`) appears in CAVEAT. |

> The trace is what makes Version 2 trustworthy. In Version 1 you could only read the model's words. Now you can see the exact SQL and the exact rows it used.

## 12 The limitation, and what Version 3 changes

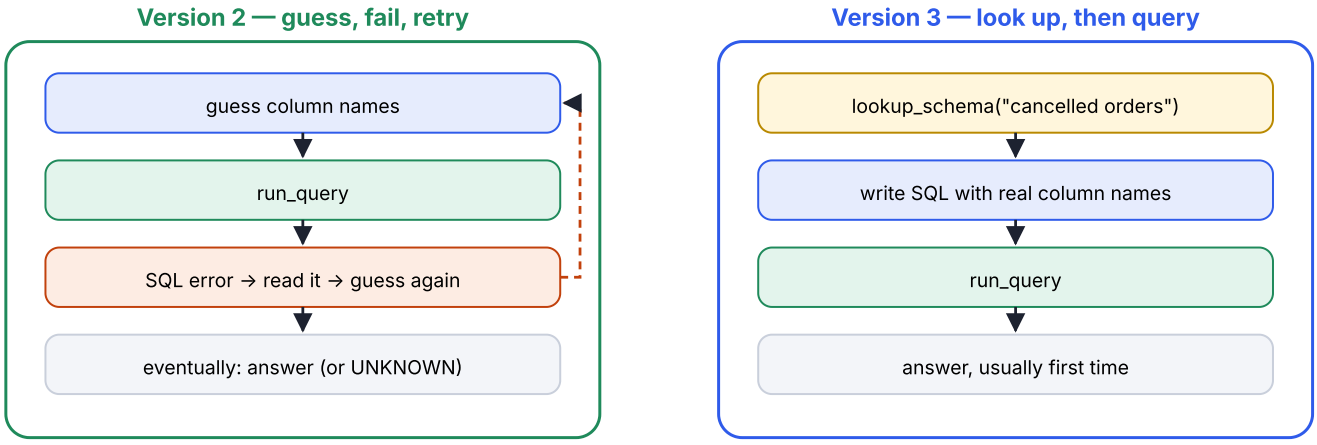



Figure 12. Version 2 learns the schema by failing. Version 3 looks it up first.

Version 2 can run SQL, but it still does not know the schema. That causes three problems:

- **Wasted passes.** Each wrong column name costs a model call and a database round trip.
- **Silent wrong answers.** A guess can *run* and still be wrong. If `status = 'cancelled'` exists but cancellations are really recorded as `'canceled'`, the query returns 0 rows and no error.
- **No business definitions.** "Revenue" or "average order value" needs a definition the database cannot give. Error messages only report broken SQL, not a wrong definition.

| | Version 2 | Version 3 |
|---|---|---|
| How it learns the schema | from SQL errors and `information_schema` | `lookup_schema` → Pinecone `nlq-schema` |
| Similar solved questions | — | `lookup_examples` → `nlq-examples` |
| Business definitions | — | `lookup_common` → `common` |
| Table list in the prompt | yes | removed — the agent looks it up |

## 13 Key takeaways

1. **A tool is a function plus a schema.** `@tool` builds the schema from the name, the docstring and the type hints. The model sees only the schema.
2. **The model asks; Python acts.** The model writes a tool call. The `tools` node runs it, and `tool_call_id` links the result to the request.
3. **The graph is `start → model ⇄ tools → end`.** After each model reply, tool calls send it to `tools`; no tool calls send it to the end.
4. **Every loop pass is a model call.** A clean run takes two passes. Each error adds at least one more.
5. **Return errors as text.** Then a failure becomes the next observation, and the model can repair its SQL.
6. **Fetch one extra row** to know whether results were cut off, and say so in the output.
7. **Safety is layered:** prompt, gate, read-only transaction, timeout, and the database role. Only the role is a real boundary.
8. **Check answers against the trace:** the number must appear in an observation, and SOURCE must match the SQL that ran.

> **Next — Version 3:** the agent gets memory. Three lookup tools search a Pinecone knowledge base for schema, solved examples and business definitions before any SQL is written.